

Counting Versus Testing the Destination: A Simulation Study of Layered Defences Against SMS OTP Flooding and Pumping

Naveed Ahmed^{a,*}

^a*Affiliation to be completed before submission, Saudi Arabia*

Abstract

Endpoints that send one-time passwords by SMS are abused to waste money (flooding) or to earn part of the termination fee (SMS pumping). We evaluate in simulation a twelve-step defence that draws trust from app attestation, signed sessions and verification outcomes, against simpler designs and 25 attacker profiles. Attacks with a signature in the request are contained at little cost to simulated users; a residential attacker with farmed CAPTCHA scores is rationed by refusing half of first-time users. On the destination, short-window counters of SMS sends per number block, selected at a benign service target, leaked 6 to 48 messages against four concentrated 20-minute pumpers at 0.06 to 0.43 points of attack-free completion, where at the busiest density the default sequential tests of verification outcomes leaked 19 to 452. Under a second predeclared protocol on seeds and workloads held out from selection, the frozen counter met its leakage and benign-cost claims, leaked no more than the tests in all but 13 of 144 cells, all against an instantly verifying carrier, and failed both service-under-attack claims when the pumper shares blocks with real users. For attacks of up to an hour, unfitted estimates whose crossing is algebraic, not a measured switching point, ordered the policies' mean leakage in 18 of 18 decisive boundary cells with the counter's cold-start bound and 17 with its warm-start bound: the counter loses its advantage against paced or widely spread pumpers.

Keywords: SMS pumping, artificially inflated traffic, one-time password, rate limiting, sequential tests, fraud prevention

*Corresponding author.

Email address: optimistbs@gmail.com (Naveed Ahmed)

1. Introduction

Most online services verify a phone number by sending it a code by SMS, from an endpoint reachable before any account exists, at a tenth of a cent to twenty cents a message. A flooder makes the service send to numbers it does not own; a pumper sends to a range whose operator shares the termination fee, turning the service’s SMS budget into its income [1]. Twitter’s owner said in 2023 that pumping had cost the company about 60 million dollars a year [2], and Huh et al. found as many attack requests as genuine ones in three months of one large service’s logs [3].

We study the defences of a registration flow: a baseline of seven controls (v1) and a twelve-step redesign with attested platforms, signed sessions, number checks before a message is paid for, reputation from whether codes were entered, a five-response risk score and a hard budget. We ask in simulation which attacks it contains against simpler designs, at what cost in completed registrations (RQ1); which layer is responsible (RQ2); when outcome-based reputation separates an attacker from simulated users (RQ3); what rationing costs when it does not (RQ4); and what destination policies cost legitimate users, with and without an attack (RQ5).

Because a pumper is paid only on its partner’s ranges, the study centres on the destination: under a predeclared comparison a plain counter of sends per number block leaked less than our sequential tests of code outcomes, and a second predeclared protocol mapped where that holds and fails. We contribute an analysis of eight gaps in the baseline, the design and an open implementation, measurements of where reputation stops separating and what rationing depends on, both destination-policy comparisons with their failures and collateral damage, and unfitted estimates of when counting leaks less than our tests; Sections 7.1–7.4 give context.

2. Background and related work

Revenue-share fraud. In international revenue share fraud, traffic is generated to an advertised number range and the termination fee is split along the route [4]. Sahin and Francillon’s 3.14 million test numbers from twelve providers show the range to be the fraudster’s managed asset [5].

33 *Detecting artificially inflated SMS.* Huh et al. trained gradient-boosted trees
 34 on 9.4 million SMS authentication logs from nineteen countries, reaching
 35 89.6% recall at 0.19% false positives on web traffic [3]. About 73% of web
 36 attacks sent four or fewer requests per identity a day, defeating per-entity
 37 limits, and destination signals were the strongest: about 92% zero conversion
 38 for attacks against 21% for genuine traffic. Their detector needs labelled
 39 traffic and acts per request. Their data show that destination aggregation
 40 is informative; we add a controlled comparison of two label-free ways to act
 41 on it per block, from send counts or code outcomes, and of their cost to
 42 users sharing a block. Theirs was not reproduced, so the two are not ranked.
 43 Zalisky and Odarchenko count A2P sends per prefix (a number less its last
 44 three digits) in fixed or rolling five-minute windows and quarantine a prefix
 45 for 24 hours past a day or night threshold [6]. On metadata of two confirmed
 46 incidents fixed windows blocked 95% and 63% of the traffic, and 58% of a
 47 legitimate marketing campaign; counting unique recipients blocked 2% of the
 48 campaign and none of the first incident. Theirs is a send counter judged on
 49 real traffic by messages blocked; ours compares a counter with outcome tests
 50 at a service target, in simulation, by what users sharing a block lose; the
 51 numbers are not comparable.

52 *Defences in practice.* Twilio’s Fraud Guard blocks regions known for pump-
 53 ing [7], and its SMS Pumping Risk Score advises friction from 60 and refusal
 54 above 90 [8], so graded destination action is established; no product was
 55 benchmarked. CAPTCHA solving is a labour market at about a dollar per
 56 thousand [9], open where machine solvers fail [10]; client puzzles [11] were
 57 not evaluated. Residential proxy addresses rotate within minutes [12], and
 58 carrier-grade NAT makes address reputation on cellular networks “all but
 59 infeasible” [13].

60 *Identity and change detection.* One entity can present many identities [14];
 61 a signed session gives continuity, not scarcity. Our block tests are Wald’s
 62 sequential tests [15] run as Page’s cumulative sums [16]. Quickest change de-
 63 tection minimises delay under false-alarm constraints [17], and a CUSUM’s
 64 parameters follow from the error rates it must meet [18]; windowed statistics
 65 and confidence sequences [19] were not evaluated, so ‘testing’ below means
 66 our conversion and speed tests only. Step 7 resembles risk-based authentica-
 67 tion [20]; attacks on the code once sent are a separate problem [21, 22].

68 3. Baseline and threat model

69 The baseline, v1, sends a code to any unregistered number behind seven
70 controls: reCAPTCHA v3 on web requests only, one message per number per
71 minute, five requests per address per minute, a country allow-list, a body-
72 length limit, a proxy block and per-platform caps; bulk and test paths skip
73 some checks. Table 1 lists eight assumptions behind these rules and how an
74 attacker within the threat model below breaks each; the gaps follow from the
75 rules, not observed abuse.

Table 1: v1’s assumptions and how an attacker within the threat model breaks them.

Gap	v1 assumes	How it fails
A	A platform header says which app sent the request	Sending the header reaches every exemption
B	Five requests per address per minute bound an attacker	A fresh residential address per request
C	A number given is worth sending to	Generated numbers cost as much as real ones
D	An allowed country is a safe destination	Premium ranges inside it cost far more
E	Per-key caps bound total spend	A wide attack under every cap
F	Check-then-increment is a rate limit	Two racing instances both pass
G	Bulk and test flags are internal	Neither needs stronger authentication
H	Each check says which check failed	Thresholds can be probed one at a time

76 *Threat model.* The attacker wants as many messages sent as possible, or
77 sent to numbers that pay it. It buys residential proxies and human-like
78 reCAPTCHA tokens (about 0.3 cents each), mints fingerprints, generates
79 numbers and knows the thresholds. A pumper also has a colluding operator’s
80 ranges, where it may enter codes, report deliveries as failed, build verified
81 history, spread over blocks and buy challenge solutions. It cannot forge an
82 attestation, use ranges its partner does not terminate, or make a stranger
83 enter a code.

84 4. Design

85 No message is paid for before every check passes, and cheap checks run
86 first. Trust comes from evidence the client cannot forge, an attestation, a

87 server-signed session and the fate of earlier codes, though such evidence can
 88 be manufactured (Section 7.8).

89 4.1. The pipeline and the risk score

90 A request passes the steps of Table 2 until one refuses it. Step 0 takes the
 91 platform only from a valid App Attest or Play Integrity result. Each limit and
 92 reservation is one atomic Redis operation, but the enclosing steps are not one
 93 transaction: on the normal path a refused request gives back its per-number
 94 claims and counter slot, and the hourly budget is reserved only for a message
 95 about to be enqueued, a reservation rather than a completed dispatch; a
 96 crash between them can leak reservations (Section 6). Address limits and
 97 source caps count requests, except that a request Step 11 challenges gives
 98 its source-cap count back. Every outcome after Step 0 except the challenge
 99 returns one HTTP 200 body after at least 400 ms, a floor that cannot hide
 100 longer work.

Table 2: The twelve steps (the v1 gaps of Table 1 they address in brackets). FB is the feedback loop.

Step	Check
0	Proxy block; platform from attestation, never from the header [A]
1	Signed session bound to a fingerprint; 3 requests per session per 10 min [B]
2	Limits per address, /24 and ASN, lower for hosting ASNs [B, F]
3-4	reCAPTCHA v3 threshold and origin check, web only
5	Number intelligence: cost class, sequential numbers, liveness; reads the block’s verdict stage [C, D, G]
6	Body length, excluded numbers; test bypass needs a credential [G]
7	Risk score and tier: allow, delay, challenge, downgrade, block [H]
8	Per-number backoff, 60 s doubling to 1 h, five a day
9	Per-source, platform and country caps with an adaptive multiplier
10	Operating mode: elevated above 80% of the hourly budget, emergency at 100% [E]
11	Reserve hourly count, spend and any destination slot; write state; enqueue; uniform response [E, H]
FB	Outcomes per key; destination-block tests; denylist of fine keys

101 Step 7 computes

$$s = \sum_i w_i x_i + w_c \frac{\max(0, \theta - r_{\min})}{\theta} + w_s f + w_m i - w_v v_n - w_f v_f, \quad (1)$$

102 where the x_i are request signals (CAPTCHA score, hosting ASN, fingerprint
 103 age, session requests, country mismatch, number class), $r_{\min}$ is the lowest con-
 104 version ratio over the request’s keys (address to destination block), $\theta = 0.3$,
 105 $w_c = 25$, f and i flag a flooded key and one whose codes are entered within
 106 seconds, and v_n , v_f mark a number and fingerprint with verified history;
 107 other weights are 5 to 25 [23]. Tiers start at 20, 40, 60 and 80: *allow*;
 108 *delay* (a doubling wait); *challenge* (-20 when solved); *downgrade* (push or
 109 WhatsApp); *block*.

110 4.2. The feedback loop

111 A send resolves as *verified* when its code is entered, *failed* after five wrong
 112 attempts or two minutes past the delivery receipt, and *undelivered* without a
 113 receipt in sixty seconds or with a failure receipt; only the first two feed rep-
 114 utation. A positive receipt after a negative one reopens the send; a late ver-
 115 ification reclassifies it, but an issued verdict stands. Each transition records
 116 its effects and its time in the same compare-and-set, and a recovery sweep
 117 finishes a dead process’s work; each effect type applies at most once, and
 118 exactly once if the sweep runs within 20 minutes. Reputation counts land
 119 in the transition’s hour and a block keeps its latest outage failure time, but
 120 a block’s statistic accumulates in processing order and a verdict is dated at
 121 the event that completes the crossing, so a late event can backdate a verdict
 122 that contains later evidence, moving its expiry and the next test’s start (by
 123 ten minutes in one diagnostic). Simulated runs apply events in time order;
 124 concurrent live instances may not, since a transition is committed before its
 125 effects apply, so the same dependence holds live unless a block’s effects are
 126 serialized.

127 4.3. Destination policies

128 A pumper is confined to its partner’s ranges but can rotate numbers
 129 and blocks within them. Destination policies are keyed on the 8-digit block
 130 (10 000 numbers), so protection weakens as a pumper spreads.

131 *Sequential tests.* The conversion statistic assumes users verify with $p_\ell = 0.8$
 132 and a flooder with $p_a = 0.1$: a failure adds $\ln \frac{1-p_a}{1-p_\ell} = 1.50$, a verification
 133 $\ln \frac{p_a}{p_\ell} = -2.08$. The speed statistic assumes a machine enters a verified code
 134 within five seconds with $q_a = 0.9$ and a person with $q_\ell = 0.2$, since autofill
 135 removes the keystroke [24, 25]; it adds 1.50 or -2.08 . A verdict is issued,
 136 and the statistic reset, when either exceeds $h = \ln 1000 = 6.91$. The floor

137 $S_t = \max(-ch, S_{t-1} + \text{increment})$ sets the memory: unbounded c is Wald’s
 138 test, $c = 0$ Page’s cumulative sum; the default is $c = 1$. A first verdict makes
 139 the block’s first-time clients pass a challenge for an hour; a second within the
 140 hour moves them off SMS, refusing those without another channel.

141 *Counters.* A counter limits SMS sends per block in a daily or short refilling
 142 window. Step 11 reserves a slot and decides the stage in one atomic operation
 143 when an SMS is about to go out, so concurrent requests cannot skip the
 144 challenge. Past the limit a graded counter challenges first-time clients, and
 145 past twice the limit moves them off SMS; a refusing counter refuses. Clients
 146 with verified history are exempt from both policies, so a code entered on a
 147 pumper’s number can buy that number an exemption. A token bucket of the
 148 same rate (burst 4 or 12; unlike RFC 2697’s excess bucket [26], its challenge-
 149 tier bucket refills independently) and the counter combined with the tests
 150 were also explored.

151 4.4. Capacity controls

152 Step 9 multiplies its caps by 0.25 to 1.5 according to a baseline job com-
 153 paring each key with its expected rate, learned from the pipeline’s hourly
 154 counters; without history the static cap stands. Verified history avoids the
 155 adaptive reduction, not the static caps or the budget, which bounds SMS
 156 count and spend, not vendor or fallback costs.

157 5. Leakage estimates and a heuristic crossover

158 We compare the SMS sent to a pumper’s numbers over an attack of
 159 $T \leq 60$ minutes, by a pumper that solves no challenge and holds no verified
 160 history. Against the sequential tests, one reaching B blocks with N requests
 161 at rate λ leaks about

$$L \approx \min(N, kB + \lambda\tau), \quad k = \lceil h/1.50 \rceil = 5, \quad (2)$$

162 before its first verdict, where τ , the time a send stays unresolved, is the
 163 resolution timeout plus the delivery delay plus half the timeout worker’s
 164 period (2.55 minutes) if the carrier never verifies, and seconds if it verifies at
 165 once. Judged, such a pumper is held for the verdict’s hour, so for $T \leq 60$
 166 Equation (2) also estimates the attack’s total; it ignores prior credit and
 167 outage suspension. A carrier verifying a share σ of codes with human-like

168 delay gives the conversion statistic an expected increment of $\sigma(-2.08) + (1 -$
 169 $\sigma)(1.50)$, zero at $\sigma^* = 0.42$; above it the statistic drifts away from a verdict.
 170 A counter of q sends per block opens a window with a block's first reserved
 171 SMS and closes it after the window length. A pumper that finds its blocks'
 172 counters empty (a cold start) sends at most

$$L_c \leq \min(N, qBw), \quad w = \lceil T/\text{window} \rceil, \quad (3)$$

173 whatever its carrier does with the codes. If a legitimate send has already
 174 opened a window on block b , using $c_b \geq 1$ slots, the pumper can also take
 175 the remaining $(q - c_b)^+$ before at most w further windows open, so in general

$$L_c \leq \min\left(N, qBw + \sum_{b \text{ open}} (q - c_b)^+\right) \leq \min(N, B(qw + q - 1)). \quad (4)$$

176 Solving challenges doubles these bounds, and verified history exempts a client
 177 from them. While neither policy's estimate reaches N , the cold-start bound
 178 lies below the tests' estimate when

$$B(qw - k) < \lambda\tau, \quad (5)$$

179 and a warm start adds the residual quota to the left side. Equation (5)
 180 is the algebraic crossing of two simplified quantities, neither an empirically
 181 established switching point nor a necessary condition: other limits can hold
 182 either policy below its estimate. A carrier that games outcomes may never
 183 be judged, so the tests' leakage approaches N .

184 In the code every send takes a slot, verified clients' included. If a block's
 185 sends arrive as a Poisson stream with mean m per window and all reach the
 186 counter, a window holds $1 + X$ of them, $X \sim \text{Poisson}(m)$, and the idealized
 187 share beyond the quota is $E[(1 + X - q)^+]/(1 + m)$: 7.3% at $m = q/2$ and
 188 27% at $m = q$ for $q = 4$. It is not a challenge rate: exempt clients take slots
 189 without challenge, and challenge failures, the second tier and earlier gates are
 190 ignored. The estimates use only the rules, the configuration and the attack's
 191 rate; what an excess send costs a user needs the simulation.

192 6. Implementation and evaluation method

193 *Implementation.* About 4000 lines of Python implement the pipeline on in-
 194 memory and Redis backends. Up to release 2.8.3 the simulator's in-memory
 195 store let a sorted set expire a fixed time after its first write, whereas Redis

196 keeps it while it is written to, so in simulation the outage detector and Step 5
 197 lost recent observations. Release 2.9.0 follows Redis in every rule compared,
 198 checked by differential tests, and every study was rerun with it; earlier re-
 199 leases repaired seven replay and concurrency defects, each with a regression
 200 test. Of 453 tests, 19 use a real `redis-server`, 18 testing specific interleav-
 201 ings of two instances, not all. Step 11 is still not one transaction: a crash
 202 after its reservations leaks reserved slots or resolves the send as undelivered.
 203 The implementation is a research prototype.

204 *Simulation.* The pipeline runs against scripted vendor fakes on a simulated
 205 clock. After a warm-up of ten minutes (30 or 130 for pumping), attacks
 206 run over legitimate traffic at 20 requests per minute. Simulated users verify
 207 80% of codes after a lognormal delay (median 22s; 2s for the 20% using
 208 autofill); 70% of numbers are reachable on WhatsApp; 90% of users complete
 209 a challenge; 20% are returning account holders with a stable fingerprint, a
 210 trusted number and verified history, so results for returning users describe
 211 trusted identities only. Separate random streams give all scenarios at a seed
 212 the same users.

213 Three behavioural parameters have published counterparts: genuine con-
 214 version (one provider’s 68% or more [27]; ours 60 to 90%) and the never-
 215 converting destinations and identity churn of Section 2; the rest are assump-
 216 tions. Attackers (Table 3) draw pool size, rate and CAPTCHA class per
 217 seed, which explains 92% [80, 98] of the variance of the farm’s leakage in
 218 a nested design (ten configurations, three seeds each). Brackets are 95%
 219 percentile-bootstrap intervals, paired per seed for differences; they describe
 220 variation under this generator, not uncertainty about real traffic.

221 *Two protocols and their chronology.* A supplementary map (`results/`
 222 `study_map.md`) gives each study’s seeds. The first protocol, committed
 223 before its runs, fixed the matched comparison and a robustness study over
 224 twelve Latin-hypercube points (conversion 0.6–0.9 and five other behavioural
 225 parameters) and a held-out family (60 to 120 attack requests a minute, 50 000
 226 to 500 000 addresses, heterogeneous legitimate traffic), with claims C1–C5
 227 that hold if true in 90% of cells, a benchmark convention, not a confidence
 228 level. The matched comparison changes only the destination policy and, at
 229 three block densities, selects on tuning seeds the least leakage summed over
 230 four concentrated pumpers among settings at or above the *tuning target*,
 231 the default’s completion minus 0.5 points (the full sequential grid only at

Table 3: The 25 attacker profiles by capability, and the destination policies each faced: S, sequential tests; C, counters; B, block-limit. Names and parameters: `results/attacker_profiles.md`.

Capability	Profiles	Policies
A signature in the request (5)	One address; datacenter rotation abroad; a premium-rate prefix; numbers walked upward; the farm with an unattested iOS header	S, B
Looks like a person (7)	Residential bot; farmed CAPTCHA with fresh or aged fingerprints; one 48-hour-old browser profile	S, B
	Low and slow, 2 or 10 requests/min over 20 ASNs; datacenter rotation buying challenge solutions	S
Games outcomes or trust (8)	Pumpers on elevated-cost or standard ranges verifying within 1 s, or 60% after 30 s	S
	Threshold-aware and receipt-faking carriers; three trust builders (500 identity/number pairs verify for 10 or 30 min, then flood)	S, C
Concentrated on the destination (5)	Four pumpers on three blocks (carrier never verifies, verifies within 1 s or 60% after 30 s, or buys challenges); a poisoner on users' blocks	S, C
Spread (generated)	3 to 300 blocks, at full rate or at most the counter's quota per block	S, C

the busiest). The robustness study's first run failed C1 in 32 of 108 cells because attack and no-attack runs drew different users; we fixed the pairing, a fix that followed a result, and reran every study. The second protocol (claims K1–K5) was written after the matched comparison had selected three counters and committed before its runs; six of its seed values had served other robustness points, never selection. Both were rerun unchanged with 2.9.0 (9 919, 7 925 and 240 runs, each carrying its code hash); against 2.8.0, total leakage changed in 855 of 17 964 comparable runs, but the principal selections, eligibility decisions and claim counts did not; one auxiliary setting (false alarms matched at unbounded credit) moved from $T1000$ to $T300$ under both protocols (artifact change report).

Designs and metrics. Seven designs: **v1**, **v1-corrected** (attested platform), **budget-only** (sessions, static caps, budget), **block-limit** (a refusing counter of five sends per block per day), **conversion-only** and **speed-only** (v2 minus one block test), and **v2**. A seed is *contained* when leakage stays at most 5% of the attack rate from some minute to the window's end, for five minutes or more. *Attributable* loss compares the same requests with the policy enforced and absent, net of the reverse, in points of offered users: on attack-free traffic it is the *benign cost*, on the attacked trace the *extra harm* relative to the attacked no-policy arm. *Degradation* compares the attacked trace with attack-free operation under the same policy.

Table 4: Leaked share (% , mean of per-run shares) of about 600 attack requests in twenty minutes, thirty seeds, under v1, the block-limit design (BL) and v2 with source caps lifted, and under v2 with caps on, with the first-time users refused and all users completing (%). All designs, with intervals: supplementary Table S1 ([results/design_table.md](#)).

Attacker	caps lifted: leaked			caps on, v2		
	v1	BL	v2	leaked	refused	completed
Single client	5.6	0.3	0.3	0.3	1.1	79.1
Datacenter rotation	69.0	69.0	0.0	0.0	1.1	79.1
Premium pumping	99.4	1.0	0.0	0.0	1.1	79.1
Sequential numbers	99.4	1.0	11.5	10.7	3.5	77.5
Reused profile	99.5	99.5	22.5	17.4	8.6	74.2
Residential bot	37.2	37.2	37.2	29.9	16.2	69.6
Farmed CAPTCHA	99.5	99.5	99.5	45.6	54.0	45.5
Aged fingerprints	99.5	99.5	99.5	45.6	54.0	45.5
Forged header	100.0	99.5	99.5	45.6	54.0	45.5

7. Results

7.1. RQ1: what is contained, and at what cost

Datacenter rotation leaks nothing under v2 because it scores in the challenge tier and declines challenges; Step 5 refuses the premium range, and the session cap holds the single client to two messages (Table 4); in all 108 robustness cells these attacks leaked under 5% at completion within two points of the paired no-attack run (claim C1). The walk leaks 61 [53, 70] messages before its block’s verdict (block-limit: five), the reused profile 89 [83, 95].

The farm leaks 99.5% with caps lifted under every design; v2’s adaptive cap holds it to 229 [215, 244] messages, 46%, by refusing 54% [47, 60] of first-time users, so 45.5% complete instead of 76.4% under v1: rationing (claim C5 held in 36 of 36 cells).

7.2. RQ2: conditional effects of each layer

In the ablation grid (90 effects, each conditional on all other layers, unadjusted intervals; artifact), removing the adaptive caps lets the farm leak 310 [194, 421] more messages; the risk engine, feedback loop and block key hold the walk by 166, 152 and 121 messages, and the loop holds the reused profile by 138. The key acts only through the loop (interaction -121 [-141 , -101]).

272 *7.3. RQ3: where reputation stops separating*

273 A farmed-CAPTCHA attacker uses an address and fingerprint once and
 274 shares its country, prefix and residential ASN with every legitimate user, so
 275 its failures penalize everyone: over an hour with caps lifted (five seeds), up
 276 to twice the legitimate rate 99 to 100% of it leaks, since a shared key stays
 277 above 0.3 conversion until failures outnumber legitimate sends about 1.7 to
 278 one. At ten times, 70% leaks while 17% of legitimate users are challenged and
 279 3.2% refused (claim C2, at least 90% leaked with caps lifted, held in 34 of 36
 280 cells). No single change to the risk score separates better: a fresh-fingerprint
 281 weight of 40 stops the attacker only by challenging 47% of legitimate users.
 282 In the settings run, reputation separated an attacker only on keys its sends
 283 dominated [14].

284 *7.4. RQ4: what rationing costs, and what it depends on*

285 When separation fails, the cap is the lever: over ten seeds the default
 286 lets 47% of the farm’s requests through and refuses 41% of all legitimate
 287 users, the tightest setting 20% and 64%. Both assume an oracle job, told
 288 the legitimate rate and run every minute; run every five minutes or hourly, it
 289 lets 75 to 96% through. Learning the rate from the pipeline’s counters, the
 290 deployed job leaks 193 [161, 229] messages with a clean three-week profile,
 291 340 [239, 441] more on a cold start, 106 [62, 151] more with a stale profile
 292 and 45 [10, 77] more with a poisoned one.

293 *7.5. Pumping against the sequential tests*

Table 5: Concentrated pumpers on three blocks, caps lifted, ten seeds: messages leaked of about 574 requests in twenty minutes; seeds contained; mean minutes to containment (contained seeds) and to the first verdict (seeds with one); codes entered.

Carrier, design	Leaked (msgs)	Contained (seeds)	Containment (min)	First verdict (min)	Codes entered
never verifies: v1	570 [419, 736]	0/10	–	–	0
v2 default	90 [71, 114]	10/10	3.5 [3.1, 3.9]	3.0	0
verifies within 1 s: v2 default	19 [17, 21]	10/10	1.4 [1.1, 1.7]	0.4	19
verifies 60% after 30 s: v2 default	452 [365, 539]	0/10	–	11.6	272
v2 default, 60 min	1 067 [839, 1 310]	2/10	47.0 [42.0, 52.0]	25.5	–
buys challenges: v2 default	106 [87, 127]	10/10	4.3 [3.8, 4.9]	3.0	0

294 In Table 5 the speed test catches a carrier verifying within a second at 19
 295 messages, while the carrier verifying 60% with human-like delay is contained

in none of ten twenty-minute runs: its first verdict comes at minute 11.6 on average and 452 messages leak; in an hour 1 067 leak and two of ten runs are contained, at minute 47.0 on average. Claim C3 (containment with at most 30% leaked) held in 36 of 36 cells.

Across 18 spread configurations leakage follows the blocks touched: with 30 blocks five to seven runs in ten are still contained. Unsaturated, Equation (2) errs by 1.6 to 7.9 messages per run (1 to 13%, ten configurations); saturated, it overstates by up to 91.

7.6. Destination policies: where the counter wins, and where it fails

Table 6: Matched comparison, first protocol, 200 blocks (144 legitimate sends per block a day), tuning seeds 100–102, evaluation seeds 0–9: twenty-minute leakage (messages) by never-verifying (NV), instant (IV), human-like (HL) and challenge-buying (CB) carriers; from 24-hour attack-free runs at 65% conversion, completion, benign cost (points), users challenged and verdict events a day. Tuning target: 63.95%.

Setting (selected as)	NV	IV	HL	CB	Compl.	Benign	Chall.	Events
none	319	566	570	405	64.13%	–	1.53%	0
sequential <i>T</i> 1000, <i>c</i> 1 (default)	90	19	452	106	64.12%	0.01	1.64%	6.5
sequential <i>T</i> 300, <i>c</i> 0 (selected at tuning target)	88	14	126	98	63.77%	0.37	6.54%	334.3
counter, graded, 4 per 10 min (selected)	24	24	24	48	64.07%	0.06	2.38%	0
counter, graded, 160 per day (selected daily)	319	409	408	405	64.11%	0.02	1.87%	0

At a benign service target. Under the first protocol (Table 6, Figures 1 and 2), the sequential setting selected at the tuning target holds the human-like carrier to 126 messages only by raising 334 false verdict events a day, and misses the target by 0.18 points on seeds 0–9. The graded counter of four sends per block per ten minutes leaks 24 against each outcome-reading carrier and 48 against the challenge buyer, at 0.06 points; counters selected at the sparser settings leak 6 to 18 at 0.15 to 0.35 points (up to 0.43 at 80%). With the full grid and density-specific attacks (E5) the rule selects the same counters, and so do other objectives on the tuning runs (worst-case leakage, a first-time-loss constraint, each attacker alone, 2 000 random mixes), except the instant verifier alone and 3 mixes at 200 blocks; on fresh seeds the counter leaked least of the winners against every attacker but the instant verifier (25, against 15 and 16). On seeds 300–309 the 1 000-block counter falls 0.12 points below its target, yet no seed-and-conversion cell fails the paired check (the evaluation default minus 0.5 points).

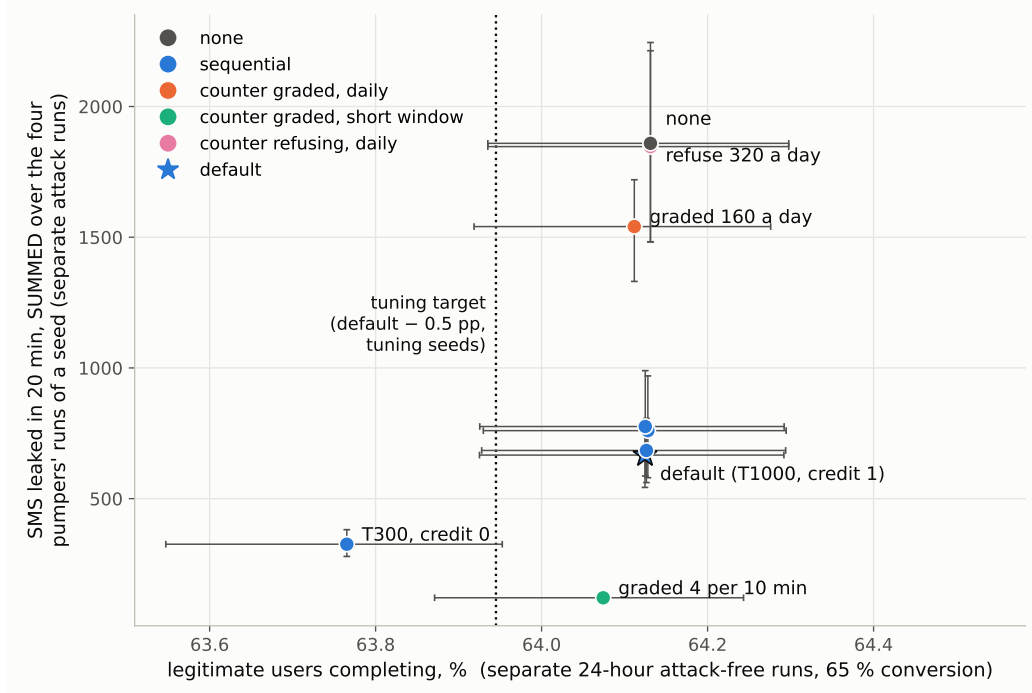


Figure 1: Matched comparison, first protocol, 200 blocks, evaluation seeds 0–9: each setting’s completion in 24-hour attack-free runs at 65% conversion against its 20-minute leakage summed over the four concentrated pumpers’ runs of a seed (not one combined attack). Bars: 95% bootstrap intervals over seeds. Dotted line: the tuning target. Unlabelled points: sequential settings matched to the default’s false alarms at other credits (key: `results/figure1_key.md`).

320 *Under a second protocol.* Frozen and run on the held-out family (Table 7),
321 the counter leaked 33, 34, 33 and 62 messages against the four carriers,
322 where the default leaked 287, 36, 344 and 287 and no policy 451 to 1857, at
323 a benign cost of 0.13 [0.09, 0.17] points, and failed both service-under-attack
324 claims: 13 of the 17 cells that broke K3 lie at the three points with the most
325 legitimate traffic, where users and pumper compete for one quota; K5 failed
326 in 51 cells over 11 of 12 points. The counters selected for sparser traffic cost
327 4.7 and 5.8 points here even without an attack: the quota must match the
328 traffic.

329 In Table 8, against the attacked no-policy arm the counter cost all users
330 nothing on average and gained the attacked blocks’ users 3 points on average,
331 since without a destination policy the pumper exhausts their blocks and the

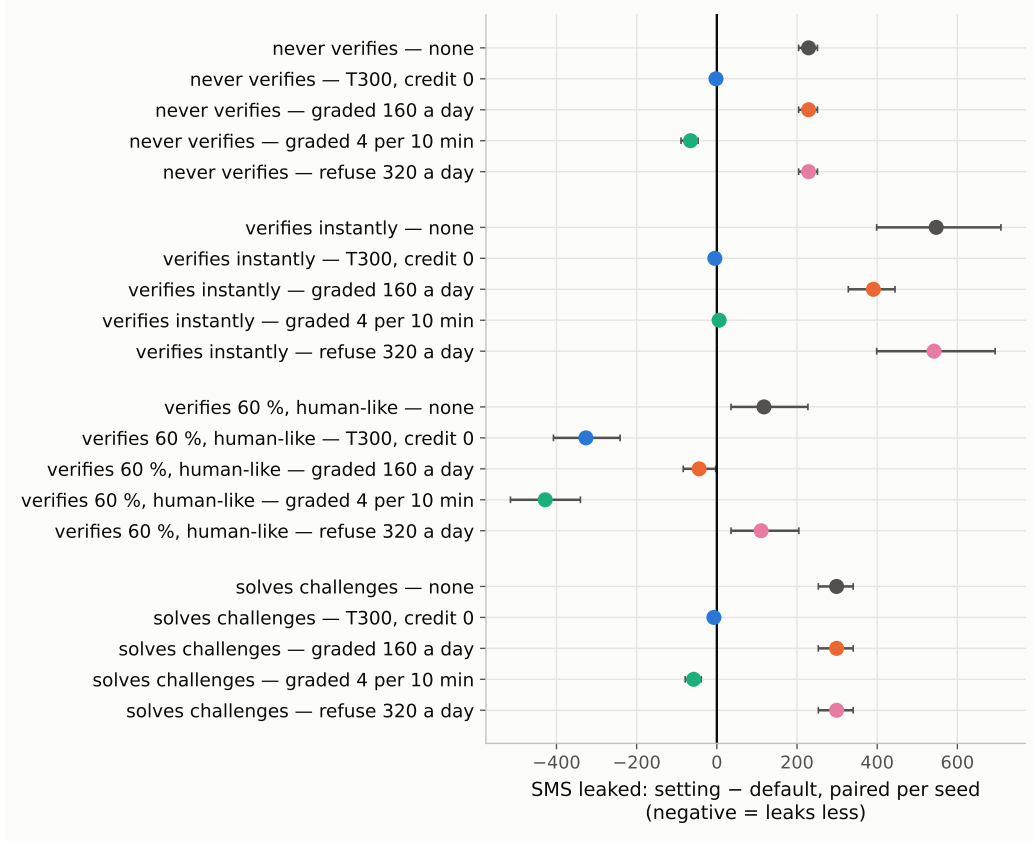


Figure 2: Matched comparison, first protocol, 200 blocks, evaluation seeds 0–9, 20-minute attacks: for each selected setting and each concentrated pumper, the setting’s leakage minus the default’s, paired per seed (negative: leaks less), with 95% bootstrap intervals over seeds.

hourly budget; with the human-like carrier alone it cost them 1.9. Against
attack-free operation every evaluated policy still left those users 5 to 9 points
worse off on average (2 to 12 per carrier; 8 with none): none shields a block
an attacker shares. Counter and tests together, explored outside the frozen
claims, leaked least summed over the four carriers but not against the instant
verifier (28; $T300$, $c0$: 22), and under the poisoner (Table 9) cost all users
most of the counters and buckets and leaked more than $T300$, $c0$. A token
bucket with a burst of 4 spared the poisoner’s users only by letting almost
everything through; with two untuned settings neither limiter dominates,
and we rank only the implemented policies. Returning users, all simulated

Table 7: The second protocol’s claims for the principal counter (graded, 4 per 10 min) on the held-out family: 12 shifted points, held-out attack rates and pools, three seeds each, a 20-minute pumper on three blocks real users share. A claim holds if true in 90% of cells; cells share workloads and seeds and are not independent. IV: the instant verifier’s cells.

Claim	Statement	True / cells	IV	Result
K1	Leaks at most 10% of attack requests	144/144 (100%)	36/36	holds
K2	Attack-free attributable loss at most 0.5 points	36/36 (100%)	–	holds
K3	Attributable loss under attack at most 0.5 points	127/144 (88%)	32/36	fails
K4	Leaks no more than the default sequential tests	131/144 (91%)	23/36	holds pooled, not for IV
K5	Attributable loss under attack at most the default’s plus 0.25 points	93/144 (65%)	23/36	fails

Table 8: Attacked service (E1) under a pumper on three blocks that real users share: 200 blocks, 60-minute attacks, seeds 300–309, four carriers. Leaked: SMS to the attacker’s numbers, summed over the four carriers’ runs of a seed, which can hide a carrier on which a policy loses (per carrier: artifact). Harm: net completions lost, in points, averaged over the carriers; negative: a gain. Brackets, on leakage and the attacked blocks only: 95% bootstrap intervals over seeds.

Policy	Leaked (SMS)	Extra harm against attacked, no policy		Degradation against attack-free, same policy		Benign cost
		all users ($\approx 1\,200$)	attacked blocks (≈ 18)	all users	attacked blocks	
none	5 509 [4 409, 6 567]	–	–	0.19	8.0 [5.1, 10.7]	–
sequential, default	1 149 [953, 1 340]	–0.07	–1.0 [–2.8, 0.8]	0.10	7.0 [4.7, 9.1]	0.02
sequential $T300$, $c0$	397 [312, 481]	0.13	0.9 [–1.3, 3.3]	0.12	8.9 [6.0, 11.7]	0.20
counter, 4 per 10 min	403 [386, 418]	–0.05	–3.2 [–5.0, –1.1]	0.07	4.8 [2.9, 6.7]	0.07
counter + sequential	190 [179, 201]	–0.04	–3.1 [–5.1, –0.7]	0.07	4.9 [3.0, 6.8]	0.08
token bucket, burst 4	424 [412, 437]	–0.12	–3.1 [–5.1, –0.7]	0.07	4.9 [2.8, 7.0]	0.00
token bucket, burst 12	545 [531, 559]	–0.11	–2.1 [–4.3, 0.2]	0.08	5.9 [3.3, 8.8]	0.00

342 with verified history, lost nothing.

Table 9: Attacked service (E1) under a poisoner flooding all 200 blocks: 60-minute attacks, seeds 300–309. Extra harm: net completions lost, in points of all users (with 95% bootstrap intervals over seeds) and of first-time users.

Policy	Leaked (SMS)	Extra harm against attacked, no policy	
		all users	first-time users
none	1 976	—	—
sequential, default	1 357	1.09 [0.59, 1.59]	1.36
sequential $T300$, $c0$	858	2.55 [2.04, 3.07]	3.20
counter, 4 per 10 min	1 662	0.72 [0.44, 1.02]	0.91
counter + sequential	1 286	1.43 [0.88, 1.97]	1.79
token bucket, burst 4	1 877	0.12 [0.04, 0.23]	0.16
token bucket, burst 12	1 974	0.00 [0.00, 0.00]	0.00

Attack-free (Figure 3a), the counter costs a block’s users nothing at one send per ten minutes, 2.3 points at four and 5.5 at eight, far below the idealized share beyond the quota (27% at four), since most challenged users complete or switch to WhatsApp; without WhatsApp eight sends cost 13.2. A 30-minute launch at three times the normal rate costs its blocks’ users 16 points. Over 60 minutes (Figure 3c) the default leaked 110, 144 and 242 messages on 3, 10 and 30 blocks, against 105, 139 and 240 from Equation (2) ($\lambda\tau \approx 90$), and the counter 71, 240 and 679 against its cold-start bound of 72, 240 and 720. Once 7 messages sent, exempt, to real users’ trusted numbers are set aside, no counter run that started cold (44 of 100) exceeded Equation (3); three warm ones did, by 5 or 6 messages, and none exceeded Equation (4). With these unfitted inputs Equation (5) ordered the policies’ mean leakage in 18 of 18 decisive cells (two ties at N), and with the warm-start term in 17, missing where the estimates differ by 0.8 messages (30 blocks, 20 minutes; measured difference 6.6 [−27.2, 42.4]); per seed, 76 of 78 decisive comparisons agree. The algebraic crossing lies near five blocks for an hour and 30 for twenty minutes, where the nearest measured differences (three and 30 blocks) do not exclude zero; it is not a measured switching point. Paced to the quota, a pumper leaks about as much under the counter as with no policy (71 against 72 on three blocks in an hour), and 23 under the tests.

Against adaptive attackers (E3) the counter held the threshold-aware carrier, the receipt faker and the concentrated trust builder to 24 to 36 messages (502 to 665 under the default), but not trust builders on random numbers

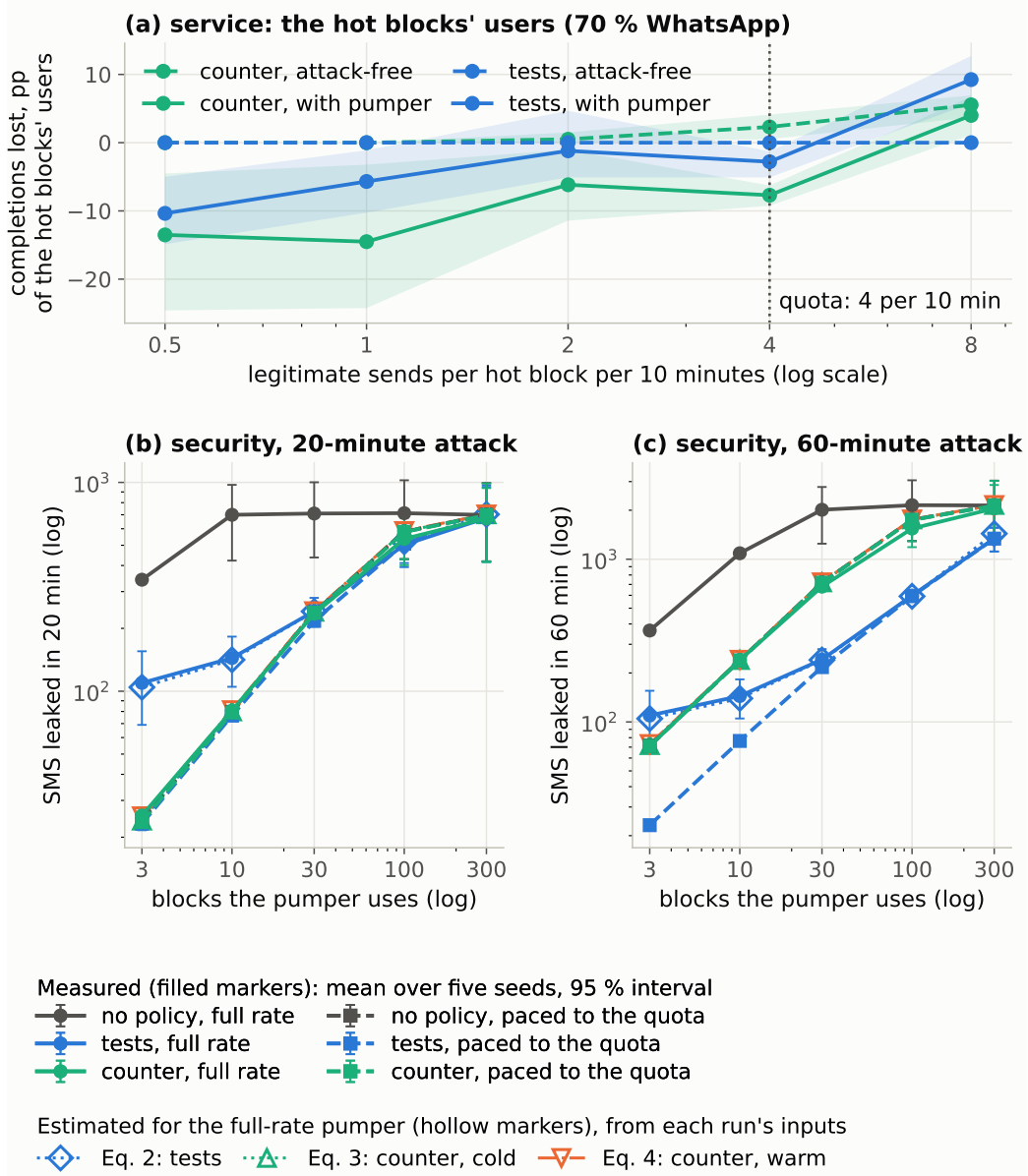


Figure 3: The counter's operating boundary (E4), seeds 300–304; lines join sampled settings and do not locate a crossover. (a) Attributable loss in points of the three hot blocks' users against their legitimate rate, attack-free and with a never-verifying pumper on those blocks. (b, c) Leakage in 20 and 60 minutes against the blocks a pumper uses, at full rate or paced, and Equations (2)–(4) for the full-rate pumper (hollow markers). All horizontal axes and the vertical axes of (b, c) are logarithmic. Bands and bars: 95% bootstrap intervals over seeds.

367 (577, as under the default); over 360 minutes the human-like carrier gained
368 81 messages an hour under the counter, 753 under the default.

369 7.7. *RQ5: what destination policies cost legitimate users*

370 On one block, 20 000 Monte Carlo trials per cell give the probability of
371 a verdict within 30 legitimate sends: 0.09% [0.06, 0.14] at 80% conversion
372 with 20% autofill, 1.9% at 65% and 19.5% at 50%. Over 24 hours the default
373 raises 0.5 verdict events a day on 200 busy blocks at 80% conversion and 6.5
374 at 65%, where they meet 30 of about 28 800 requests (claim C4 held in 18 of
375 18 cells).

376 A poisoner flooding for twenty minutes the twenty blocks legitimate users
377 concentrate on (caps lifted, ten seeds) raises 16 verdict events meeting 98
378 legitimate requests, 33 never completing; against observing without enforcing
379 on the same trace, enforcement costs 12.7 [4.6, 22.9] completions, 3.1% of
380 users (6.3% without WhatsApp; 23.9% with a 24-hour denylist). One that
381 stops after ten minutes leaves verdicts meeting 406.5 requests afterwards,
382 an attributable loss of 46.2 [16.9, 79.8]. Grading limits the harm; it caps it
383 neither at a challenge nor at the attack’s duration.

384 7.8. *Adaptive attackers, economics and overhead*

385 These scripted adaptations’ leakages are not bounds. Against v2 with
386 caps lifted, a datacenter attacker buying every challenge solution leaks 55; a
387 threshold-aware carrier and a receipt faker leak 570 each, drawing no verdict.

388 If pumpers earn a share of the 0.1422 USD retail price and pay an event-
389 level bill, they break even at a share of 0.043 under v1 and 0.054 to 0.271
390 under v2 (the instant verifier 1.310); over 360 minutes under the counter
391 the human-like carrier needs 1.048. These are break-even shares for scripted
392 attackers under assumed prices, not bounds or criminal profit.

393 On a real Redis (four vCPUs, four workers) a send costs 13.8 ms at the
394 median and 21.1 ms at the 99th percentile, and 133 requests per second pass
395 at concurrency 128 with the 400 ms floor and 50 ms vendor calls. With fast
396 vendors response times distinguished no pair of five outcome classes (small-
397 est Kolmogorov–Smirnov $p = 0.051$). With heavy-tailed vendors under an
398 adversarial mixture, 81% of sends overran the floor, and a response-time
399 threshold separated sends from refusals with 88% balanced accuracy on its
400 own run; on three new runs, thresholds fitted on one scored 95 to 99.9% on
401 the other two: a separation on that machine, not a deployment accuracy.

402 8. Discussion

403 *Lessons.* In the scripted workloads, carriers that verify with human-like de-
404 lay, fake receipts or know the thresholds defeated our outcome tests but not
405 a short-window counter. Within a verdict’s hour the counter’s allowance,
406 unlike the tests’, grows with the windows an attack spans; over longer cam-
407 paigns verdicts expire and the tests leak again. Equation (5) summarizes this
408 for pumpers that solve no challenge and hold no exemption. Its inputs are in
409 an operator’s logs; the cost of an excess send, competition under attack and
410 exemption abuse come only from the simulation. Counter and tests together,
411 with a quota set from each block’s traffic, is a hypothesis for real traffic, not
412 a recommendation.

413 *Limitations.* Everything is simulated and the calibration is assumed. Both
414 protocols were written by the author for the author’s generator, the second
415 knowing which counters had won. Some results follow from the generator
416 (a perfect premium list, a datacenter attacker that declines challenges, re-
417 turning users who always present verified history), and selection used an
418 unweighted sum of four scripted pumpers’ leakage and a mean service target
419 met on tuning seeds, which guarantees no service level elsewhere. Close rank-
420 ings rest on ten or fewer seeds, and costs that round to zero are not shown
421 equivalent. Per-client quotas within a block, carrier spend ceilings, bounded
422 exemptions, tuned token buckets and other sequential statistics lie outside
423 the evaluated policies. Vendor adapters are untested live, and the timing re-
424 sult holds only for the observer tested. Destination policies challenge people
425 for what strangers sharing their block did, and deployment under the GDPR
426 or Saudi Arabia’s PDPL needs a lawful basis and a route for refused users.
427 Replaying real traffic is the experiment this paper most needs.

428 9. Conclusion

429 In this simulation the design contains attacks that carry a signature at lit-
430 tle cost to simulated users, and rations, rather than separates, a residential at-
431 tacker that looks like a person. Counting sends on the destination leaked less
432 in aggregate than our outcome tests against concentrated pumpers, though
433 not reliably against an instantly verifying carrier; a frozen counter failed its
434 service claims when the attacker shares blocks with real users, and lost its
435 advantage against wide, paced and trust-building pumpers. Within an hour,
436 unfitted estimates ordered the policies’ mean leakage in 18 of 18 decisive

437 boundary cells with the cold-start bound and 17 with the warm-start bound,
438 missing a near tie; the crossing is algebraic, not a measured switching point.

439 **CRediT authorship contribution statement**

440 **Naveed Ahmed:** Conceptualization, Methodology, Software, Valida-
441 tion, Investigation, Writing – original draft, Writing – review & editing.

442 **Declaration of competing interest**

443 The author declares no known competing financial interests or personal
444 relationships that could have appeared to influence this work.

445 **Declaration of generative AI and AI-assisted technologies in the** 446 **writing process**

447 The author used Claude (Anthropic) for code, evaluation, drafting and
448 editing, checked numbers against the recorded results and claims against the
449 code, and takes full responsibility for the content.

450 **Data availability**

451 No human or real traffic data was used. Code, protocols (`config/`), every
452 run’s configuration and outcome and the sources of headline numbers
453 (`results/`) are at <https://github.com/nahmedpsu/OTP-DDOS>, release
454 2.9.1 (tag v2.9.1), which runs unchanged the code of release 2.9.0 (tag
455 v2.9.0, commit 9af3100) that produced the results (`code_identity.md`);
456 the 2.8.0 results are in `results/historical_2.8`. These files are the
457 supplementary material.

458 **References**

459 [1] MITRE Corporation, Resource hijacking: SMS pumping, technique
460 T1496.003, MITRE ATT&CK, [https://attack.mitre.org/techn](https://attack.mitre.org/techniques/T1496/003/)
461 [iques/T1496/003/](https://attack.mitre.org/techniques/T1496/003/), accessed 2026-10-01.

- [2] Commsrisk, Elon Musk says Twitter lost \$60mn a year because 390 telcos used bot accounts to pump A2P SMS, <https://commsrisk.com/elon-musk-says-twitter-lost-60mn-a-year-because-390-telcos-used-bot-accounts-to-pump-a2p-sms/>, trade press; the figure is the company’s own claim (2023).
- [3] J. H. Huh, H. Shin, S. Ahn, H. Yi, J. Cho, T. Kim, M. Lim, N. Choi, Preventing artificially inflated SMS attacks through large-scale traffic inspection, in: 34th USENIX Security Symposium, USENIX Association, Seattle, WA, 2025.
- [4] M. Sahin, A. Francillon, P. Gupta, M. Ahamad, SoK: Fraud in telephony networks, in: 2017 IEEE European Symposium on Security and Privacy (EuroS&P), IEEE, 2017. doi:10.1109/EuroSP.2017.40.
- [5] M. Sahin, A. Francillon, Understanding and detecting international revenue share fraud, in: Network and Distributed System Security Symposium (NDSS), Internet Society, 2021. doi:10.14722/ndss.2021.24051.
- [6] M. Zaliskyi, M. Odarchenko, Artificially inflated traffic in A2P messaging: Window-based detection models and prefix-level analysis for fraud mitigation, Science-Based Technologies 69 (1) (2026) 89–98. doi:10.18372/2310-5461.69.20735.
- [7] Twilio Inc., Verify fraud guard, <https://www.twilio.com/docs/verify/preventing-toll-fraud/sms-fraud-guard>, accessed 2026-10-01.
- [8] Twilio Inc., SMS pumping risk score (Lookup v2), <https://www.twilio.com/docs/lookup/v2-api/sms-pumping-risk>, accessed 2026-10-01.
- [9] M. Motoyama, K. Levchenko, C. Kanich, D. McCoy, G. M. Voelker, S. Savage, Re: CAPTCHAs – understanding CAPTCHA-solving services in an economic context, in: 19th USENIX Security Symposium, USENIX Association, 2010.
- [10] A. Dionysiou, E. Athanasopoulos, SoK: Machine vs. machine – A systematic classification of automated machine learning-based CAPTCHA solvers, Computers & Security 97 (2020) 101947. doi:10.1016/j.cose.2020.101947.

- 493 [11] D. Dean, A. Stubblefield, Using client puzzles to protect TLS, in: 10th
494 USENIX Security Symposium, USENIX Association, 2001.
- 495 [12] X. Mi, X. Feng, X. Liao, B. Liu, X. Wang, F. Qian, Z. Li, S. Alrizah,
496 L. Chen, Y. Qian, Resident evil: Understanding residential IP proxy
497 as a dark service, in: 2019 IEEE Symposium on Security and Privacy
498 (S&P), IEEE, 2019. doi:10.1109/SP.2019.00011.
- 499 [13] P. Richter, F. Wolsing, A. Chandrasekaran, R. Padmanabhan, M. All-
500 man, R. Bush, A. Feldmann, A multi-perspective analysis of carrier-
501 grade NAT deployment, in: Proceedings of the 2016 ACM Internet
502 Measurement Conference (IMC), ACM, 2016. doi:10.1145/298744
503 3.2987474.
- 504 [14] J. R. Douceur, The Sybil attack, in: Peer-to-Peer Systems (IPTPS
505 2002), Vol. 2429 of LNCS, Springer, 2002, pp. 251–260. doi:10.100
506 7/3-540-45748-8_24.
- 507 [15] A. Wald, Sequential tests of statistical hypotheses, The Annals of Math-
508 ematical Statistics 16 (2) (1945) 117–186. doi:10.1214/aoms/1177731
509 118.
- 510 [16] E. S. Page, Continuous inspection schemes, Biometrika 41 (1/2) (1954)
511 100–115. doi:10.2307/2333009.
- 512 [17] V. V. Veeravalli, T. Banerjee, Quickest change detection,
513 arXiv:1210.5552 (book chapter) (2012).
- 514 [18] NIST/SEMATECH, CUSUM control charts, section 6.3.2.3, e-handbook
515 of statistical methods, [https://www.itl.nist.gov/div898/handbook](https://www.itl.nist.gov/div898/handbook/pmc/section3/pmc323.htm)
516 [/pmc/section3/pmc323.htm](https://www.itl.nist.gov/div898/handbook/pmc/section3/pmc323.htm), accessed 2026-10-02.
- 517 [19] S. R. Howard, A. Ramdas, J. McAuliffe, J. Sekhon, Time-uniform, non-
518 parametric, nonasymptotic confidence sequences, The Annals of Statis-
519 tics 49 (2) (2021) 1055–1080. doi:10.1214/20-AOS1991.
- 520 [20] S. Wiefeling, L. Lo Iacono, M. Dürmuth, Is this really you? An empirical
521 study on risk-based authentication applied in the wild, in: ICT Systems
522 Security and Privacy Protection (IFIP SEC 2019), Vol. 562 of IFIP Ad-
523 vances in Information and Communication Technology, Springer, 2019,
524 pp. 134–148. doi:10.1007/978-3-030-22312-0_10.

- 525 [21] C. Mulliner, R. Borgaonkar, P. Stewin, J.-P. Seifert, SMS-based one-
526 time passwords: Attacks and defense, in: Detection of Intrusions and
527 Malware, and Vulnerability Assessment (DIMVA 2013), Vol. 7967 of
528 LNCS, Springer, 2013, pp. 150–159. doi:10.1007/978-3-642-39235-
529 -1_9.
- 530 [22] H. Siadati, T. Nguyen, P. Gupta, M. Jakobsson, N. Memon, Mind your
531 SMSes: Mitigating social engineering in second factor authentication,
532 Computers & Security 65 (2017) 14–28. doi:10.1016/j.cose.2016.
533 09.009.
- 534 [23] N. Ahmed, OTP Guard: a layered SMS validation, risk-scoring and
535 rate-limiting pipeline against OTP flood abuse (version 2.9.1), <https://github.com/nahmedpsu/OTP-DDoS>, tag v2.9.1, results produced by
536 2.9.0 (commit 9af3100), whose executable code 2.9.1 keeps; 2.8.0 results
537 (commit 1c277a0) in results/historical_2.8 (2026).
- 539 [24] Google LLC, SMS retriever API overview, <https://developers.google.com/identity/sms-retriever/overview>, accessed 2026-10-01.
540
- 541 [25] Web Incubator Community Group, WebOTP API, <https://github.com/WICG/WebOTP>, accessed 2026-10-01.
542
- 543 [26] J. Heinanen, R. Guerin, A single rate three color marker, RFC 2697,
544 Internet Engineering Task Force (1999). doi:10.17487/RFC2697.
- 545 [27] Twilio Inc., Verify API, <https://www.twilio.com/en-us/user-authentication-identity/verify>, accessed 2026-10-03.
546